

c# Programmieren (c) HTL-Leonding

BattleShip

Lehrziele

- mehrdimensionale Arrays
- Datei Lesen/Schreiben
- Datenkapsel

Aufgabenstellung

BattleShip (oder **Schiffe versenken**) ist ein Spiel für zwei Spieler. Dabei wird abwechselnd versucht, die auf einem 10x10 platzierten Schiffe des Gegners zu erraten. Gewonnen hat der Spieler, der als erster alle Schiffe erraten konnte.

siehe https://de.wikipedia.org/wiki/Schiffe_versenken

Das gesuchte Programm soll das Definieren und Validieren des Spielfeldes (fix 10x10) unterstützen. Die zu platzierenden Schiffe sind in einer Csv Datei abgelegt. Das Programm prüft, ob die richtige Anzahl von Schiffen im Csv enthalten sind und ob daraus ein gültiges "Kampfgebiet" erstellt werden kann.

Folgende Spielregeln müssen eingehalten werden:

- Die Schiffe dürfen nicht aneinander stoßen.
- Die Schiffe dürfen nicht über Eck gebaut sein oder Ausbuchtungen besitzen.
- Die Schiffe dürfen auch am Rand liegen.
- Die Schiffe dürfen nicht diagonal aufgestellt werden.
- Jeder verfügt über insgesamt 12 Schiffe (in Klammern die Größe):
 - ein Schlachtschiff (5 Kästchen)
 - zwei Kreuzer (je 4 Kästchen)
 - drei Zerstörer (je 3 Kästchen)
 - vier U-Boote (je 2 Kästchen)
 - zwei Torpedoboote (je 1 Kästchen)

Hinweis: es gibt verschiedene Definitionen der Anzahl bzw. erlaubten Größen. Dieses sollte im Programm leicht zu verändern sein.

Programmablauf

Schreiben Sie ein Programm mit dem Namen **BattleShip** .

- Die Csv Datei wird entweder durch einen Kommandozeilenparameter angegeben oder, wenn dieser fehlt, wird der Dateiname "**Ships.csv**" verwendet.
- Alle Schiffe werden von der Csv Datei in ein Array der Datenkapsel (**Ship**) gelesen.

- Anschließend wird geprüft, ob mit den gelesenen Schiffen ein gültiges (Schlacht-)feld erstellt werden kann.
- Ist die Definition gültig, wird das Feld auf der Konsole ausgegeben.
- Bei einer ungültigen Definition wird eine entsprechende Fehlermeldung ausgegeben.

Programmdesign

Achten Sie bei der Umsetzung auf ein sauberes Design Ihres Programms.

In dem bereitgestellten Programm-Template sind Unittests vorhanden. Die Unittests prüfen auch interne Programmfunktionalitäten (Hilfs-Methoden). Diese sind hilfreich bei der Umsetzung - passen Sie das Design daher so an, dass diese Methoden verwendet werden. Änderungen an den *Unitests* sind selbstverständlich nicht erlaubt.

Programminter können/sollen folgende Punkte umgesetzt sein:

Datenkapsel Ship

- **Row**
Startposition - Row - des Schiffes, 0 basiert
- **Col**
Startposition - Column - des Schiffes, 0 basiert
- **ShipSize**
Größe des Schiffes. Ein Schlachtschiff ist 5 groß.
- **IsVertical**
Vertikale- oder horizontale Ausrichtung.

Die Datenkapsel soll von der Csv Datei mit folgendem Inhalt gelesen werden können:

(Anstatt **IsVertical** wird in der Csv eine **Orientation** mit den möglichen Inhalten H oder V gespeichert)

```
Row;Col;ShipSize;Orientation
9;5;5;H
0;6;4;H
```

Methoden (BattleShip.cs)

- **Ship[] ReadFromCsv(string fileName)**
Die Csv Datei wird gelesen und der Inhalt in einem Array der Datenkapsel **Ship** gespeichert. Eine Validierung (z.B. Anzahl der Schiffe) wird NICHT durchgeführt.
- **bool[,] CreateField(Ship[] ships)**
Die Methode erstellt das (fixe Größe: 10x10) Spielfeld. Jedes durch ein Schiff belegte Feld wird mit "true" initialisiert, alle anderen Felder bleiben "false".
Es werden nur gültige Spielfelder erstellt. Im Fehlerfall gibt die Methode **null** als Ergebnis zurück.
Gültige Spielfelder sind jene Felder, die die richtige Anzahl von Schiffen aufweisen und die Schiffe korrekt platziert haben (siehe **Aufgabenstellung**).
- **void Print(Ship[] ships)**
Die Methode druck das aufgrund der übergebenen Schiffe erstellte Spielfeld auf der Konsole aus. Siehe **Bildsschirmausgabe**.

- `bool ArrangeShip(bool[,] field, Ship ship)`

Hilfsmethode, die innerhalb von **CreateField** verwendet werden kann. Dabei wird *ein* Schiff am Spielfeld platziert. Kann das Schiff nicht platziert werden (z.B. es überschneidet sich mit einem anderen), wird *false* als Ergebnis geliefert.

Hinweis: Prüfen Sie mit einer Methode (z.B. CanArrangeShip) vor dem Setzen des Schiffes, ob die Position noch frei ist.

- Verwenden sie die Konstanten im Programm

```
private const int Size = 10;
```

```
private static int[] ShouldBeAmounts = new int[] { 0, 2, 4, 3, 2, 1 };
```

Testen Sie das Programm ausführlich.

BildschirmAusgabe

```
BattleShip Validator
=====
Ships from: Ships.csv
```

```

  A B C D E F G H I J
+---+---+---+---+---+
1|x|x| |x| | |x|x|x|x|
+---+---+---+---+---+
2| | |x| | | | | |
+---+---+---+---+---+
3|x| | |x| |x|x|x| |x|
+---+---+---+---+---+
4|x| | | | | | | |x|
+---+---+---+---+---+
5|x| | | | | | | | |
+---+---+---+---+---+
6|x| | | |x| |x| |x|
+---+---+---+---+---+
7| | | | |x| | | |x|
+---+---+---+---+---+
8|x|x| | | | | | |x|
+---+---+---+---+---+
9| | | | | | | | | |
+---+---+---+---+---+
10| | |x| | |x|x|x|x|x|
+---+---+---+---+---+
```